

ARCHITECTURAL IMPLEMENTATION SPECIFICATION

CEO DASHBOARD (NO LIMITS COMPANY)

Target Persona: CEO Frontend Stack: React 18+, Next.js, Tailwind CSS Visual Ref: CEO Dashboard (Primera Página)

Este documento especifica los requerimientos técnicos y el diseño del sistema de presentación front-end para el **CEO Dashboard de NO LIMITS COMPANY**. La interfaz debe implementarse sin componentes estáticos o hardcoded.

1. SYSTEM ARCHITECTURE METADATA (FRONTEND SCOPE)

- **Frontend Stack:** React 18+, Next.js (App Router), Tailwind CSS para utilidades estéticas, Lucide React para iconografía corporativa.
- **Renderizado de KPIs:** Uso de Recharts incorporando Framer Motion o Canvas para animaciones fluidas de gráficos analíticos de alto rendimiento.
- **Integración:** Consumo directo de la RESTful Web API por medio de JSON payloads y canales reactivos en tiempo real.

2. DESIGN SYSTEM TOKENS (TAILWIND CONFIG)

La interfaz debe reflejar con precisión una estética ejecutiva oscura de alta fidelidad con los siguientes tokens:

- Background Canvas:** Deep Slate Carbon (#0f1319 a #131922)
- Card Containers:** Elevated Dark Indigo-Grey (#1b2330 con bordes desaturados en #242f42)
- Text Highlighting:** Bright Ice Blue & Clean White (ffffff, #e2e8f0)
- Accent Data Drivers:**
- Progress Tracks: Aqua Cyan / Teal gradient (#06b6d4 a #3b82f6)
 - Alert Low/Negative: Salmon Red Crimson (#ef4444)
 - Delta Positive Indicator: Emerald Mint (#10b981)

3. CORE DASHBOARD COMPONENT MATRIX

Orquestación modular del Workspace principal de la aplicación utilizando Next.js/React:

```
// components/DashboardGrid.tsx
import React from 'react';
import DailySummaryCard from './DailySummaryCard';
import AgendaTimeline from './AgendaTimeline';
import PriorityTasksList from './PriorityTasksList';
import KeyProjectsTracker from './KeyProjectsTracker';
import KpiMetricsPanel from './KpiMetricsPanel';
import UrgentNotifications from './UrgentNotifications';
import TeamActivityFeed from './TeamActivityFeed';

export default function DashboardGrid({ userId }: { userId: number }) {
  return (
    <div className="min-h-screen bg-[#0f1319] text-slate-100 p-6 font-sans">
      { /* GLOBAL HEADER BAR */ }
      <header className="flex flex-col md:flex-row justify-between mb-8 pb-4 border-b border-slate-800">
        <div>
          <h1 className="text-2xl font-bold tracking-tight text-white">CEO DASHBOARD</h1>
        </div>
      </header>

      { /* CORE FRAMEWORK WORKSPACE LAYOUT */ }
      <div className="grid grid-cols-1 xl:grid-cols-12 gap-6">
        <div className="xl:col-span-5 flex flex-col gap-6">
          <DailySummaryCard userId={userId} />
          <AgendaTimeline userId={userId} />
          <PriorityTasksList userId={userId} />
        </div>

        <div className="xl:col-span-7 flex flex-col gap-6">
          <KeyProjectsTracker />
          <KpiMetricsPanel />
          <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
            <UrgentNotifications userId={userId} />
            <TeamActivityFeed />
          </div>
        </div>
      </div>
    </div>
  );
}
```

4. EXECUTION & DELIVERY REQUIREMENTS

- **State Revalidation:** Implementar React Query (useQuery) o React Context para refrescar de forma aislada los componentes cada 30 segundos evitando estancamiento de la interfaz.
- **Loading States & Skeletons:** Desarrollar componentes tipo Skeleton que mantengan las proporciones exactas de las tarjetas visuales mientras los parámetros JSON terminen de parsear.
- **Error Boundaries:** Confinar las divisiones funcionales de la UI dentro de Error Boundaries estándar de React para aislar fallos inesperados de red o de datos.